

Componentes de JUnit5

- **JUnit Platform:** esta pieza se integra con el IDE (IntelliJ IDEA, Eclipse, etc.) y con las herramientas de construcción (Maven, Gradle, etc.) para facilitar el descubrimiento y ejecución de tests. Para ello, se introduce el concepto de Launcher que es el encargado de realizar la búsqueda, filtrado y ejecución de los tests.
- **JUnit Jupiter:** es el nuevo modelo programático para la automatización de test que proporciona JUnit5.
- **JUnit Vintage:** pues ya lo indica el adjetivo vintage. Permite retrocompatibilidad para soportar la ejecución de test desarrollados con JUnit3 y JUnit4.

Diferencias importantes entre JUnit 5 y JUnit 4

Las pruebas de **JUnit 5 (Jupiter)** se ven prácticamente iguales a las de JUnit 4, pero hay algunas diferencias que se deben tener en cuenta.

Importaciones

JUnit 5 usa el nuevo **org.junit.jupiter** paquete para sus anotaciones y clases. Por ejemplo, **org.junit.Prueba** se convierte en **org.junit.jupiter.api.Test**.

Anotaciones

La anotación `@Test` ya no tiene parámetros; cada uno de estos se ha movido a una función. Por ejemplo, para indicar que se espera que una prueba lance una excepción en JUnit 4:

```
@Test
void testThrowsException() throws Exception {
    Assertions.assertThrows(Exception.class, () -> {
        //...
    });
}
```

En JUnit 5, esto ha cambiado a:

```
@Test
void testThrowsException() throws Exception {
    Assertions.assertThrows(Exception.class, () -> {
        //...
    });
}
```

Del mismo modo, los tiempos de espera han cambiado. En JUnit 4, se veían así:

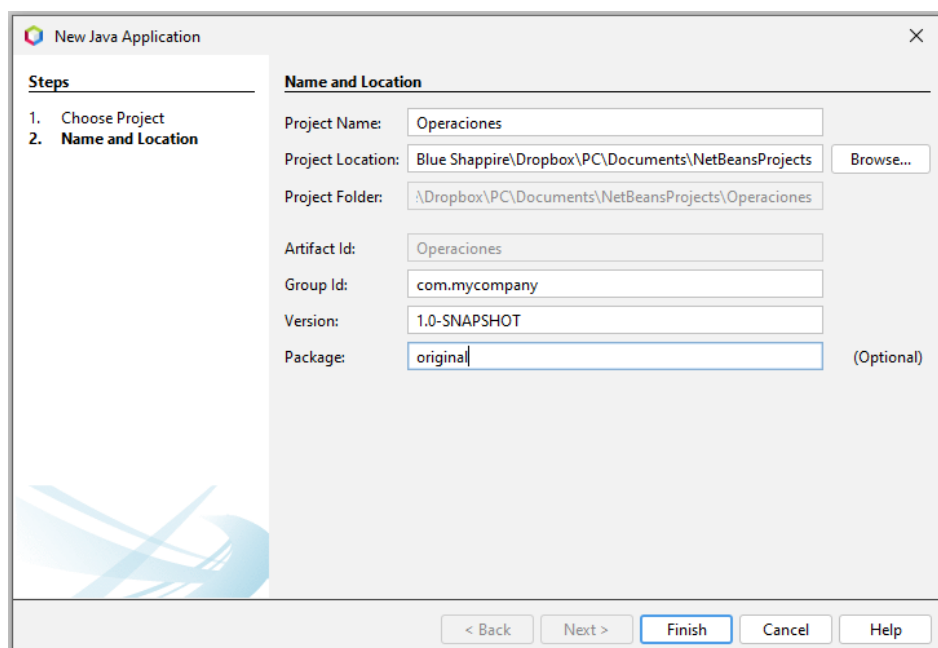
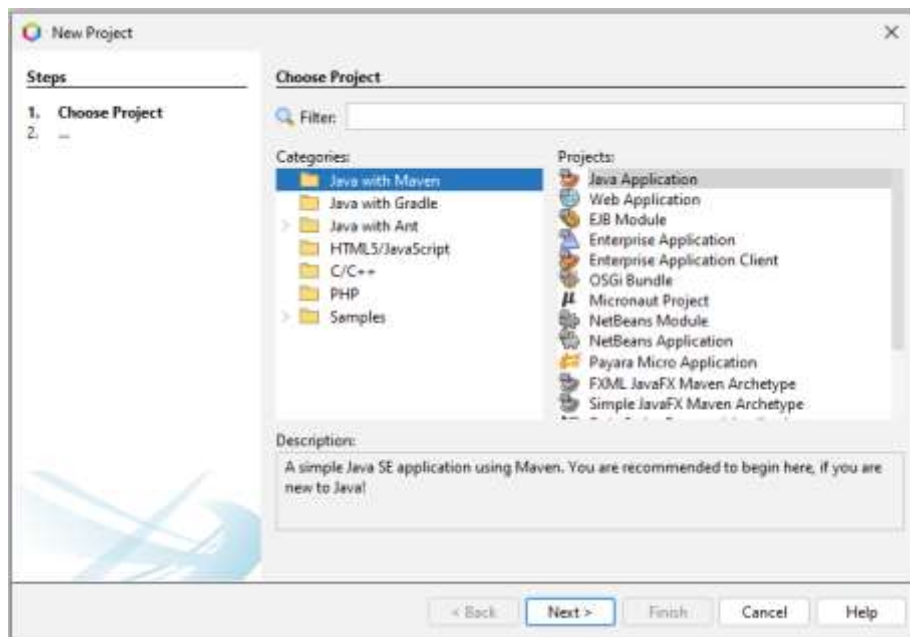
```
@Test(timeout = 10)
public void testFailWithTimeout() throws InterruptedException {
    Thread.sleep(100);
}
```

En JUnit 5, los tiempos de espera se ven así:

```
@Test
void testFailWithTimeout() throws InterruptedException {
    Assertions.assertTimeout(Duration.ofMillis(10), () ->
        Thread.sleep(100));
}
```

La información completa en: <https://junit.org/junit5/>

Podemos crear un nuevo proyecto. En este caso crearemos un proyecto usando Maven (<https://es.wikipedia.org/wiki/Maven>):



Se implementará la siguiente clase como ejemplo:

```
package original;
import javax.swing.JOptionPane;
public class Operaciones {

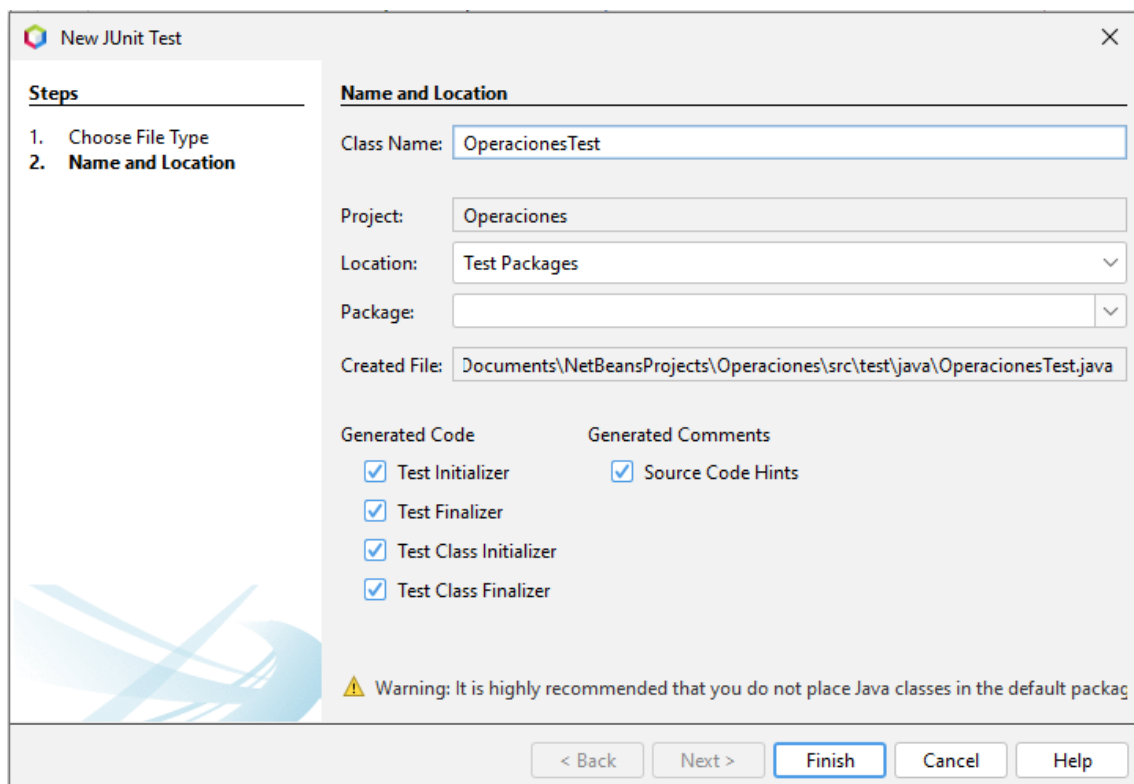
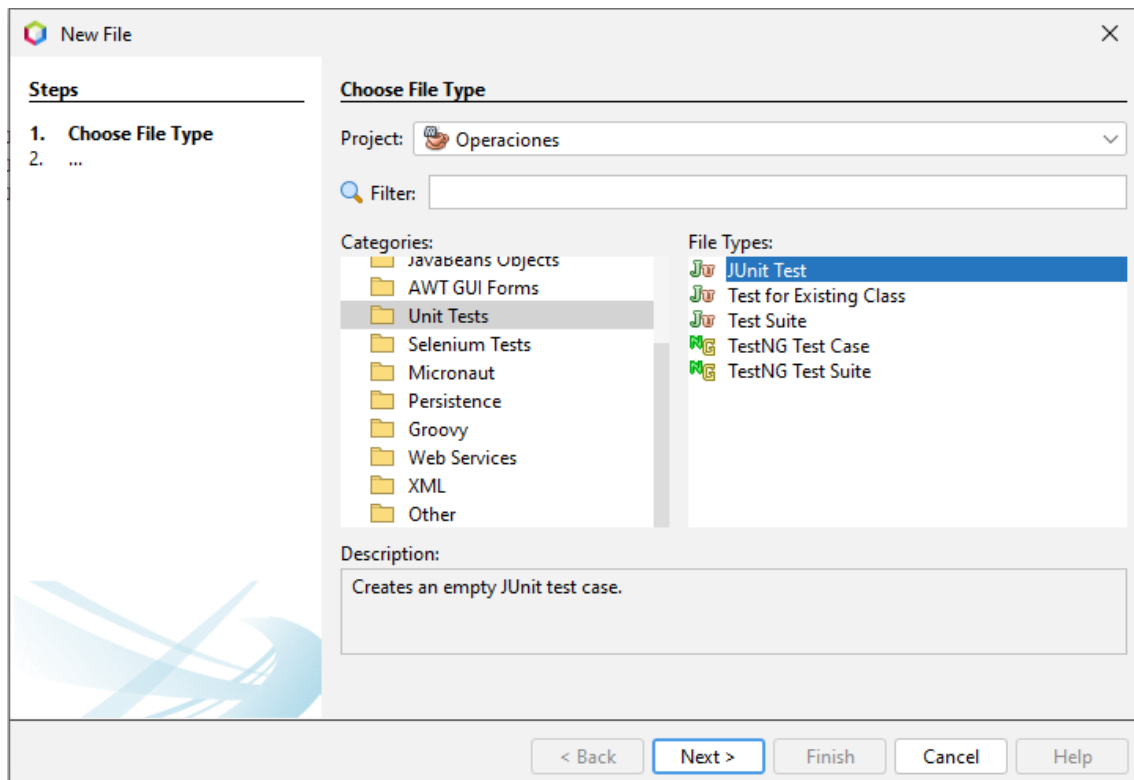
    public static float suma(float a, float b){
        return a+b;
    }

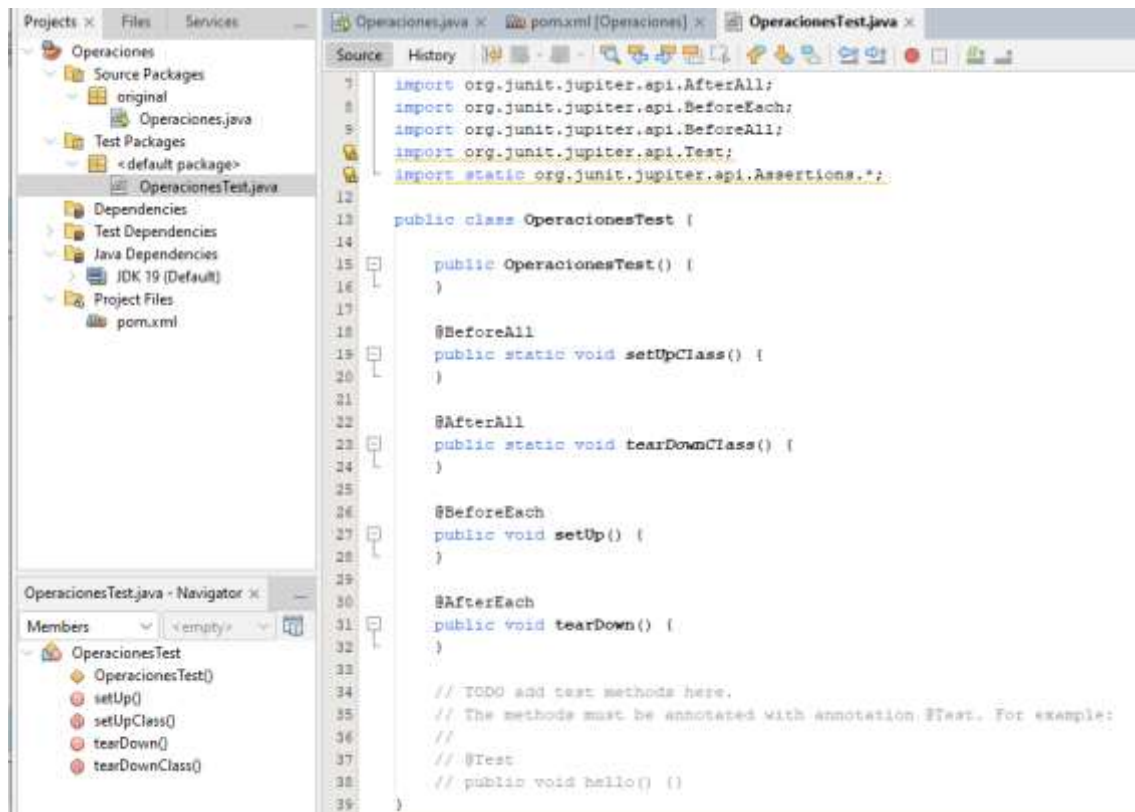
    public static float resta(float a, float b){
        return a-b;
    }

    public static float multiplicacion(float a, float b){
        return a*b;
    }

    public static float division(float a, float b){
        return a/b;
    }
    public static void main(String[] args) {
        JOptionPane.showMessageDialog(null, "Se presenta un ejemplo de calculadora básica");
        String dato = "";    int variable = 0;    float n1 = 0, n2 = 0;

        dato = JOptionPane.showInputDialog("Ingrese el primer dígito");
        n1 = Float.parseFloat(dato);
        dato = JOptionPane.showInputDialog("Ingrese el segundo dígito");
        n2 = Float.parseFloat(dato);
        dato = JOptionPane.showInputDialog("Escriba el número de la operación\naritmética
            que desea realizar:\n1. SUMA\n2. RESTA\n3. PRODUCTO\n4. DIVISIÓN");
        variable = Integer.parseInt(dato);
        switch (variable) {
            case 1:
                JOptionPane.showMessageDialog(null, n1 + " + " + n2 + " = " + suma(n1,n2));
                break;
            case 2:
                JOptionPane.showMessageDialog(null, n1 + " - " + n2 + " = " + resta(n1,n2));
                break;
            case 3:
                JOptionPane.showMessageDialog(null, n1 + " * " + n2 + " = " + multiplicacion(n1,n2));
                break;
            case 4:
                JOptionPane.showMessageDialog(null, n1 + " / " + n2 + " = " + division(n1,n2));
                break;
            default:
                JOptionPane.showMessageDialog(null, "La opción ingresada NO es válida");
                break;
        }
    }
}
```





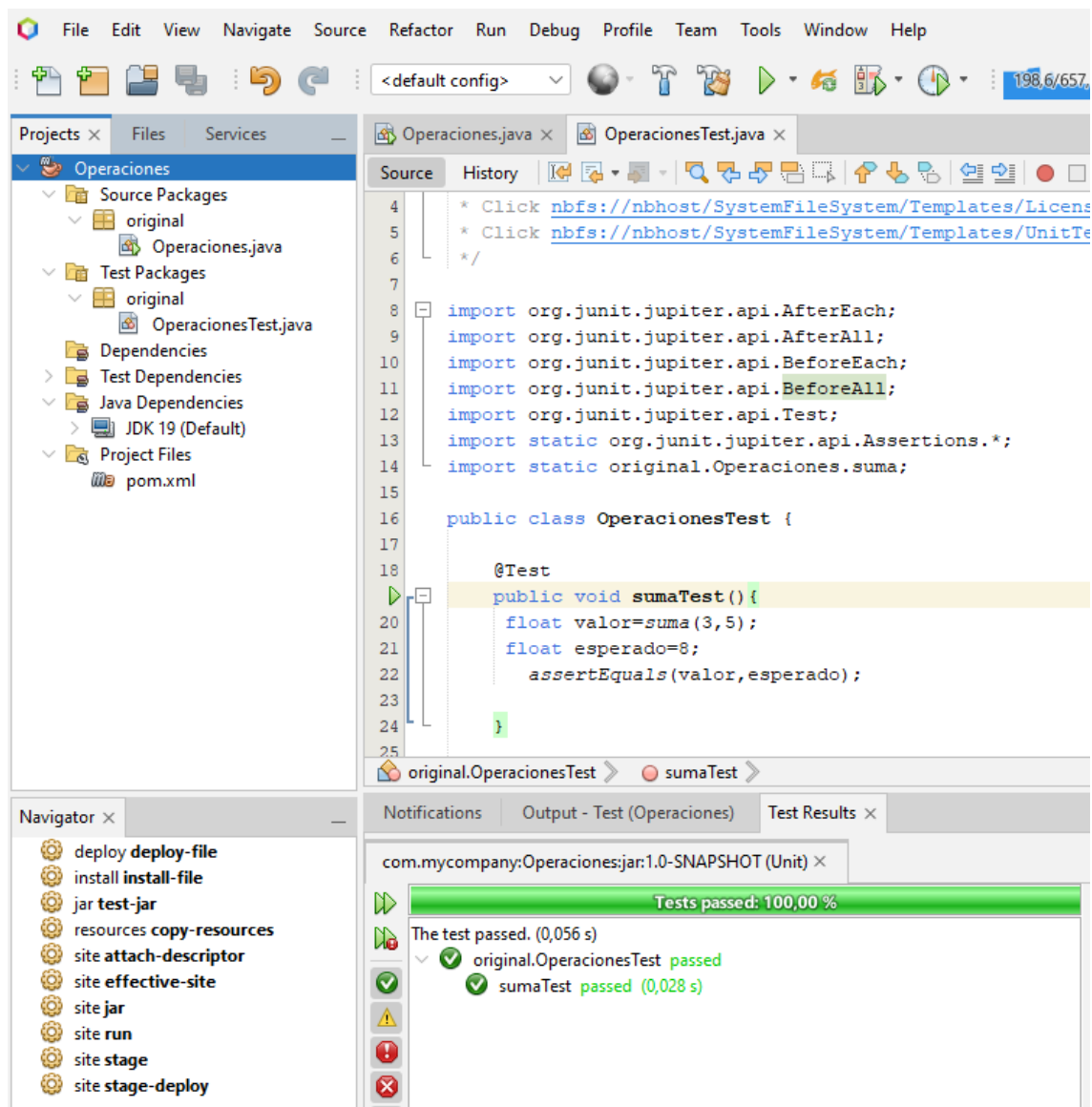
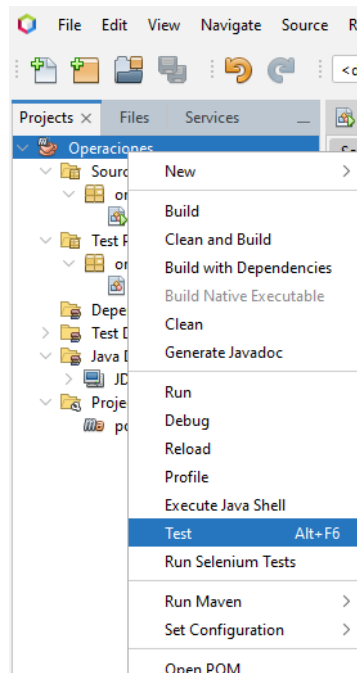
```

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
import static original.Operaciones.suma;

public class OperacionesTest {

    @Test
    public void sumaTest() {
        float valor=suma(3,5);
        float esperado=8;
        assertEquals(valor,esperado);
    }
}

```



Hay que tener en cuenta que las anotaciones han cambiado con respecto a la versión 4:

- Las aserciones están en la clase **org.junit.jupiter.api.Assertions**
- **@Before** y **@After** se deben sustituir por **BeforeEach** y **AfterEach**
- **@BeforeClass** y **@AfterClass** se deben sustituir por **BeforeAll** y **AfterAll**
- **@Ignore** deja de existir, se debe utilizar **@Disabled**
- **@RunWith** se debe sustituir por **ExtendWith**.

Annotation	Description
@BeforeEach	Denotes that the annotated method should be executed <i>before each</i> @Test , @RepeatedTest , @ParameterizedTest , or @TestFactory method in the current class; analogous to JUnit 4's @Before . Such methods are <i>inherited</i> unless they are <i>overridden</i> .
@AfterEach	Denotes that the annotated method should be executed <i>after each</i> @Test , @RepeatedTest , @ParameterizedTest , or @TestFactory method in the current class; analogous to JUnit 4's @After . Such methods are <i>inherited</i> unless they are <i>overridden</i> .
@BeforeAll	Denotes that the annotated method should be executed <i>before all</i> @Test , @RepeatedTest , @ParameterizedTest , and @TestFactory methods in the current class; analogous to JUnit 4's @BeforeClass . Such methods are <i>inherited</i> (unless they are <i>hidden</i> or <i>overridden</i>) and must be static (unless the "per-class" test instance lifecycle is used).
@AfterAll	Denotes that the annotated method should be executed <i>after all</i> @Test , @RepeatedTest , @ParameterizedTest , and @TestFactory methods in the current class; analogous to JUnit 4's @AfterClass . Such methods are <i>inherited</i> (unless they are <i>hidden</i> or <i>overridden</i>) and must be static (unless the "per-class" test instance lifecycle is used).
@Nested	Denotes that the annotated class is a nested, non-static test class. @BeforeAll and @AfterAll methods cannot be used directly in a @Nested test class unless the "per-class" test instance lifecycle is used. Such annotations are not <i>inherited</i> .
@Tag	Used to declare <i>tags</i> for filtering tests, either at the class or method level; analogous to test groups in TestNG or Categories in JUnit 4. Such annotations are <i>inherited</i> at the class level but not at the method level.
@Disabled	Used to <i>disable</i> a test class or test method; analogous to JUnit 4's @Ignore . Such annotations are not <i>inherited</i> .
@ExtendWith	Used to register custom extensions . Such annotations are <i>inherited</i> .

Methods annotated with **@Test**, **@TestTemplate**, **@RepeatedTest**, **@BeforeAll**, **@AfterAll**, **@BeforeEach**, or **@AfterEach** annotations must not return a value.

Annotation	Description
<code>@Test</code>	Denotes that a method is a test method. Unlike JUnit 4's <code>@Test</code> annotation, this annotation does not declare any attributes, since test extensions in JUnit Jupiter operate based on their own dedicated annotations. Such methods are <i>inherited</i> unless they are <i>overridden</i> .
<code>@ParameterizedTest</code>	Denotes that a method is a parameterized test . Such methods are <i>inherited</i> unless they are <i>overridden</i> .
<code>@RepeatedTest</code>	Denotes that a method is a test template for a repeated test . Such methods are <i>inherited</i> unless they are <i>overridden</i> .
<code>@TestFactory</code>	Denotes that a method is a test factory for dynamic tests . Such methods are <i>inherited</i> unless they are <i>overridden</i> .
<code>@TestInstance</code>	Used to configure the test instance lifecycle for the annotated test class. Such annotations are <i>inherited</i> .
<code>@TestTemplate</code>	Denotes that a method is a template for test cases designed to be invoked multiple times depending on the number of invocation contexts returned by the registered providers . Such methods are <i>inherited</i> unless they are <i>overridden</i> .
<code>@DisplayName</code>	Declares a custom display name for the test class or test method. Such annotations are not <i>inherited</i> .

Prueba parametrizada JUnit

La prueba parametrizada consiste en ejecutar la misma prueba una y otra vez utilizando diferentes valores. Ayuda al desarrollador a ahorrar tiempo al ejecutar la misma prueba, que solo difiere en sus entradas y resultados esperados.

Al utilizar la prueba parametrizada, se puede configurar un método de prueba que recupere datos de alguna fuente de datos. Considere una prueba sencilla para sumar diferentes números. El código puede verse así:

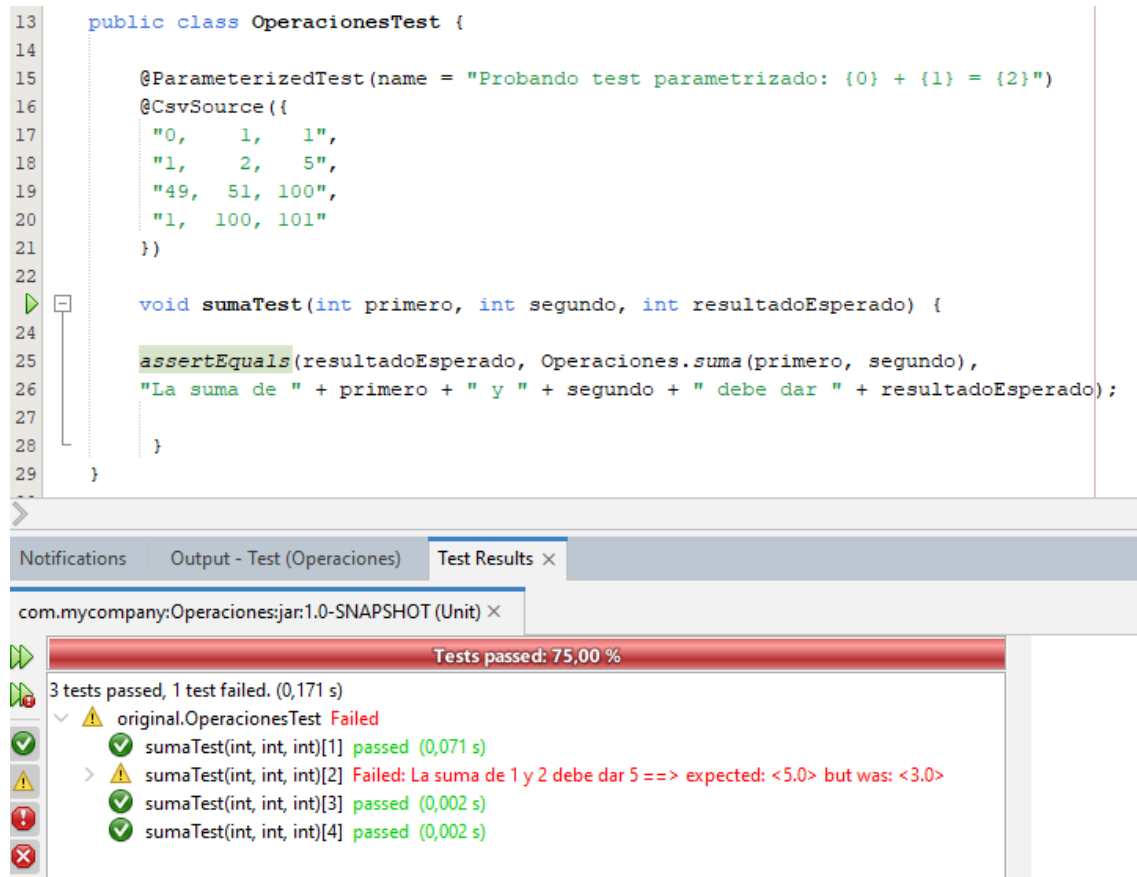
```
@Test
public void sumaTest() {
    assertEquals(20, Operaciones.suma(10.0f, 10.0f), "10 + 10 deben sumar 20");
    assertEquals(20, Operaciones.suma(20.0f, 20.0f), "10 + 10 deben sumar 40");
    assertEquals(20, Operaciones.suma(30.0f, 30.0f), "10 + 10 deben sumar 60");
}
```

Necesitamos un enfoque simple y. Al utilizar la prueba parametrizada, puede simplemente agregar un método para ingresar 10 entradas de datos y su prueba se ejecutará 10 veces automáticamente.

Pasando parámetros a nuestros tests

Esta nueva anotación simplifica muchísimo la ejecución de test con diferentes casos de uso. Para ello, podremos introducir múltiples valores y evaluar cada uno de ellos.

En el ejemplo siguiente vemos un caso muy sencillo de este tipo de tests:



Tendremos que importar:

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
```

Para ampliar un poco, recomiendo visitar:

<https://refactorizando.com/parametized-test-junit-5-ejemplos/>
<https://www.paradigmadigital.com/dev/nos-espera-junit-5/>

Ampliamos la prueba añadiendo una comprobación del tiempo de ejecución con los métodos:

```
@Test
void timeoutSuccess() {
    assertTimeout(Duration.ofMinutes(1), ()->{});
}

@Test
void timeoutFailed() {
    assertTimeout(Duration.ofMillis(1), ()->Thread.sleep(100));
}
```

Podríamos usar **assertTimeoutPreemptively** en vez de **assertTimeout**. La diferencia entre estas dos aserciones es que **assertTimeout** espera a que el código que se estaba comprobando termine, incluso nos indica por cuánto tiempo ha fallado el test. Mientras tanto, **assertTimeoutPreemptively** aborta el proceso una vez se excede el tiempo. Probando con el ejemplo anterior:

```
15 public class OperacionesTest {
16
17     @ParameterizedTest(name = "Probando test parametrizado: {0} + {1} = {2}")
18     @CsvSource({
19         "0, 1, 1",
20         "1, 2, 3",
21         "49, 51, 100",
22         "1, 100, 101"
23     })
24
25     void sumaTest(int primero, int segundo, int resultadoEsperado) {
26
27         assertEquals(resultadoEsperado, Operaciones.suma(primero, segundo),
28             "La suma de " + primero + " y " + segundo + " debe dar " + resultadoEsperado);
29     }
30
31     @Test
32     void timeoutSuccess() {
33         assertEquals(Duration.ofMinutes(1), ()->{});
34     }
35
36     @Test
37     void timeoutFailed() {
38         assertEquals(Duration.ofMillis(1), ()->Thread.sleep(100));
39     }
40 }
```

original.OperacionesTest > sumaTest >

Notifications Output - Test (Operaciones) Test Results ×

com.mycompany:Operaciones:jar:1.0-SNAPSHOT (Unit) ×

Tests passed: 83,33 %

5 tests passed, 1 test failed. (0,288 s)

- original.OperacionesTest Failed
 - sumaTest(int, int, int)[1] passed (0,063 s)
 - sumaTest(int, int, int)[2] passed (0,001 s)
 - sumaTest(int, int, int)[3] passed (0,002 s)
 - sumaTest(int, int, int)[4] passed (0,001 s)
 - timeoutSuccess passed (0,005 s)
 - timeoutFailed Failed: execution exceeded timeout of 1 ms by 108 ms

En el ejemplo se usa notación lambda de Java. Aprovechando la ocasión haremos una introducción a la notación lambda introducida desde Java 8.

Expresiones Lambda

1. Introducción

Las expresiones lambda se usan idealmente cuando necesitamos hacer algo simple y estamos más interesados en hacer el trabajo rápidamente en lugar de nombrar formalmente la función. Las expresiones lambda también se conocen como **funciones anónimas**. Su sintaxis básica se detalla a continuación:

(parámetros) -> { cuerpo-lambda }

1. El operador lambda (->) separa la declaración de parámetros de la declaración del cuerpo de la función.

2. Parámetros:

- Cuando se tiene un solo parámetro no es necesario utilizar los paréntesis.
- Cuando no se tienen parámetros, o cuando se tienen dos o más, es necesario utilizar paréntesis.

3. Cuerpo de lambda:

- Cuando el cuerpo de la expresión lambda tiene una única línea no es necesario utilizar las llaves y no necesitan especificar la cláusula return en el caso de que deban devolver valores.
- Cuando el cuerpo de la expresión lambda tiene más de una línea se hace necesario utilizar las llaves y es necesario incluir la cláusula return en el caso de que la función deba devolver un valor.

Algunos ejemplos de expresiones lambda pueden ser:

```
• z -> z + 2
• () -> System.out.println(» Mensaje 1 «)
• (int longitud, int altura) -> { return altura * longitud; }
• (String x) -> {
    String retorno = x;

    retorno = retorno.concat(» ***»);

    return retorno;
}
```

Como hemos visto las expresiones lambda son funciones anónimas y pueden ser utilizadas allá donde el tipo aceptado sea una interfaz funcional pero... **¿qué es una interfaz funcional?**

Una interfaz funcional es una interfaz con uno y sólo un método abstracto. La declaración es exactamente igual que las interfaces normales con dos características adicionales:

- Tiene un único método abstracto, como ya hemos dicho.
- De manera opcional puede estar anotada como **@FunctionalInterface**.

El motivo de que la interfaz tenga un único método abstracto es que será la expresión lambda la que proveerá de la implementación para dicho método. A continuación algunos ejemplos de interfaz funcional:

```
@FunctionalInterface
public interface Runnable {
    public abstract void run();
}
```

```
public interface MiInterfaz {
    default void saluda() {
        System.out.println("Un saludo!");
    }
    public abstract int calcula(int dato1, int dato2);
}
```

```
@FunctionalInterface
public interface Comparador {
    // Se eluden los métodos default y estáticos
    int compare(T o1, T o2);
    // El método equals(Object obj) es implícitamente implementado por la clase objeto.
    boolean equals(Object obj);
}
```

2. Tipos

Las expresiones lambda pueden clasificarse de la siguiente manera:

- Consumidores
- Proveedores
- Funciones
- Operadores Unarios
- Operadores Binarios
- Predicados

A continuación iremos detallando uno a uno y contando en qué consisten.

2.1. Consumidores

Se trata de aquellas expresiones lambda que aceptan un solo valor y no devuelven valor alguno.

```
String message -> System.out.println(message);
```

Las expresiones *BiConsumidoras*, un caso especial de las expresiones consumidoras, son aquellas que toman dos valores como parámetro y no devuelven resultado.

```
(String key, String value) -> System.out.println("Key: %s, value: %s%n", key, value);
```

2.2. Proveedores

En este caso se trata de expresiones que no tienen parámetros pero devuelven un resultado.

```
() -> return createRandomInteger()
```

2.3. Funciones

Aquellas expresiones que aceptan un argumento y devuelven un valor como resultado y cuyos tipos no tienen por qué ser iguales.

```
Order persistedOrder -> persistedOrder.getIdentifier();
```

Las BiFunciones son aquellas expresiones de tipo función que aceptan dos argumentos y devuelven un resultado.

```
(Address address, String name) -> new Person(name, address);
```

2.3.1 Operadores Unarios

Caso especial de funciones en las que tanto el parámetro como el valor devuelto son del mismo tipo.

```
String message -> message.toLowerCase()
```

2.3.2 Operadores Binarios

Igual que en el caso de los Operadores Unarios, se trata de un caso especial de funciones en las que los dos argumentos y el resultado son del mismo tipo.

```
(String message, String anotherMessage) -> message.concat(anotherMessage);
```

2.4. Predicados

Se trata de expresiones que aceptan un parámetro y devuelven un valor lógico.

```
String message -> message.length > 50
```

Como en los casos anteriores, se pueden tener BiPredicados, predicados que en lugar de tener un parámetro, tienen dos.

```
(path, attr) -> String.valueOf(path).endsWith(".js") && attr.size() > 1024
```

Ver: <https://openwebinars.net/blog/crear-expresiones-lambda-en-java/>

3. Referencias a métodos

Las referencias a los métodos nos permiten reutilizar un método como expresión lambda. Para hacer uso de las referencias a métodos basta con utilizar la siguiente sintaxis:

referenciaObjetivo :: nombreDelMetodo

```
File::canRead // en lugar de File f -> f.canRead();
```

Con las referencias a los métodos se ofrece una anotación más rápida para expresiones lambda simples y existen 3 tipos diferentes:

- Métodos estáticos.
- Métodos de instancia de un tipo.
- Métodos de instancia de un objeto existente.

Ejemplo de uso con método estático:

```
(String info) -> System.out.println(info) // Expresión lambda sin referencias.  
System.out::println // Expresión lambda con referencia a método estático.
```

Ejemplo de uso con método de un tipo:

```
(Student student, int registryIndex) -> student.getRegistry(registryIndex) // Expresión lambda  
sin referencias.  
Student::getRegistry // Expresión lambda con referencia a método de un tipo.
```

Ejemplo de uso con método de un objeto existente:

```
Student student -> getMarks(student) // Expresión lambda sin referencias.  
this::getMarks // Expresión lambda con referencia a método de un objeto existente.
```

4. Uso

Como hemos visto con anterioridad, las lambdas pueden ser utilizadas allá donde el tipo de parámetros aceptados sea una interfaz funcional. Este es el caso de muchas de las

funcionalidades que ofrece **java.util.stream**, nuevo Api que aparecio con Java 8, que permite la programación funcional sobre un flujo de valores sin estructura. Veamos algunos ejemplos:

Ejemplo 1:

```
List aList = ...
...
aList.stream()
    .findFirst(element -> element.getValue() == VALUE_TO_COMPARE)
    .ifPresent(System.out::println);
```

Mediante el código anterior hacemos uso de lambdas para:

- `element -> element.getValue() == VALUE_TO_COMPARE`: otorgamos un predicado a la función `findFirst` de la API Stream, esta función utilizará el predicado para devolver un `Optional` con el primer valor que corresponda la expresión lambda.
- `System.out::println`: establecemos un consumidor para la función `ifPresent`, de la clase `Optional`, que en caso de existir el valor, ejecutará la expresión lambda.

Ejemplo 2:

```
Map<String, Integer> map = new TreeMap<>();
map.put....
...
StringBuilder stringBuilder = new StringBuilder();
map.forEach((letter, number) -> stringBuilder.append(letter.concat(String.valueOf(number))));
System.out.println(stringBuilder.toString());
```

En el anterior código utilizamos una lambda para procesar valores, a través de un consumidor, que concatena los valores enteros que se obtienen del mapa.

Ejemplo 3:

```
try (BufferedReader reader = Files.newBufferedReader(Paths.get("SomeLines.txt"),
StandardCharsets.UTF_8)) {
    reader.lines()
        .flatMap(line -> Stream.of(line.split(WORD_REGEX)))
        .distinct()
        .map(String::toLowerCase)
        .forEach(System.out::println);
}
```

El código anterior toma un fichero, recorre sus líneas, mapea los valores obtenidos de cada línea que correspondan con la expresión regular `WORD_REGEX`, filtra aquellos que sean distintos, los pasa a minúsculas y los imprime por pantalla uno a uno.

- `line -> Stream.of(line.split(WORD_REGEX))`: Devuelve un `Stream` con los valores obtenidos de aplicar la división de la línea mediante la expresión regular. El método `flatMap` irá recogiendo los streams generados de esta manera y los convertirá a un único stream que podrá ser procesado con posterioridad.

- `String::toLowerCase`: transforma los strings del stream a minúsculas.
- `System.out::println`: ya lo vimos con anterioridad, muestra por consola cada string.

5. Conclusiones

Las expresiones lambda de Java 8 nos ofrecen varias mejoras con respecto a las versiones anteriores:

- Nos acerca a la programación funcional.
- Hace nuestro código más preciso y legible, mejorando, en consecuencia, su mantenibilidad.
- Su utilización junto con la API Stream hace más fácil la ejecución concurrente de tareas.
-

En este tutorial hemos mostrado en qué consisten y qué tipos hay, cuándo pueden utilizarse y las diferentes maneras de utilizarlas.